Deep Reinforcement Learning Vulnerability Scanner for Multi-Vulnerability Web Application Benchmarks

Hein Htet Zaw
College of Digital Innovation Technology
Rangsit University
Pathumthani, Thailand
devildog.kk@gmail.com

Karn Yongsiriwit
College of Digital Innovation Technology
Rangsit University
Pathumthani, Thailand
karn.y@rsu.ac.th

***Abstract*— Traditional vulnerability testing often relies on manual penetration testing or static heuristic scanners. While these methods are effective to a degree, they are difficult to scale and can miss complex, multi-step exploit chains. To solve this, we propose a more adaptive approach: modeling the web vulnerability discovery process as a Markov Decision Process (MDP) and training an Extended Double Dueling Deep Q-Network (Extended D3QN) to navigate it. Our configured agent incorporates five major components of the state-of-the-art Rainbow DQN algorithm. By building a custom Gymnasium environment, WebSecurityGym, we train our agent against diverse mock applications containing real-world vulnerabilities such as SQL Injection (SQLi) and Cross-Site Scripting (XSS). Guided by a Phase-Based Learning strategy that naturally progresses from reconnaissance to active exploitation, the agent learns to autonomously perform multi-step attack sequences to uncover complex vulnerabilities.**

Keywords— deep reinforcement learning, web vulnerability scanning, reinforcement learning, autonomous penetration testing, web application security

I. INTRODUCTION

Web applications have increased significantly in complexity, bringing a continuously expanding attack surface. Today's automated security tools—like Dynamic Application Security Testing (DAST)—are useful for identifying easily detectable vulnerabilities, but they often struggle when faced with complex logical flaws that require chaining multiple actions together. Real-world penetration testers excel at this because they can dynamically adapt to the target's behavior. However, relying on human experts alone is unscalable, expensive, and sometimes inconsistent across audits. There is a need for automated systems that can approximate sequential decision-making behavior observed in human testers.

Recently, Reinforcement Learning (RL) has shown strong performance at exactly this type of problem: sequential decision-making in complex environments. By setting up web security as an RL problem, we can train an agent to probe an application, parse the HTTP responses, and iteratively adjust its payloads until a vulnerability is identified.

In this work, we present a partially autonomous web vulnerability scanner driven by a Deep Q-Network (DQN) architecture. Our system leverages a custom environment, WebSecurityGym, which abstracts standard HTTP interactions into states and actions. The core component of the scanner is implemented using an Extended D3QN—a partial Rainbow DQN combining Double Q-Learning, Dueling Networks, Prioritized Experience Replay, Noisy Networks, and Multi-Step Learning— to efficiently handle a large array of different payloads. To accelerate the agent's learning process, we also introduce a Phase-Based Learning strategy that mirrors the Cyber Kill Chain, locking advanced exploits until the agent has successfully mapped the application's surface. Through extensive evaluation on multiple vulnerable mock targets, results indicate that this RL-based orchestrator demonstrates potential for scaling intelligent, adaptive penetration testing.

II. RELATED WORK

The intersection of artificial intelligence and cybersecurity has seen explosive growth, particularly in automating vulnerability analysis. This section reviews the existing literature surrounding web application security testing, the transition from manual to automated methodologies, and the emerging role of Deep Reinforcement Learning (DRL) in modern penetration testing frameworks.

A. Traditional Web Application Security Testing

Securing web applications has traditionally relied solely on well-established testing methodologies. Dynamic Application Security Testing (DAST) represents a highly active, real-time approach to identifying vulnerabilities by practically probing live applications through simulated attacks [1]. Unlike static source code analysis, DAST actively observes the application's runtime behavior, making it uniquely effective for discovering deep deployment flaws, server configuration errors, and severe runtime injection vulnerabilities [1]. In a direct comparison between manual hacking and automated scripted scanning, research proves that while automated scripts achieve high performance, only

the human eye consistently catches abstract, contextual business logic flaws [12].

Furthermore, ethical hacking and manual penetration testing inevitably rely heavily on automated scanning tools. In fact, these tools are so prevalent that forensic researchers actively study the distinct network "tool marks" left behind by various automated offensive security toolkits [13]. For instance, comparisons of widely used vulnerability scanners, such as OWASP ZAP and Nikto, demonstrate effectiveness in identifying common weaknesses across thousands of pages [2]. While these tools provide structured vulnerability analysis, they operate predominantly on rigid predefined rules and signatures. Consequently, they often struggle significantly with complex, multi-step exploits or novel attack vectors (zero-days) that require deep contextual understanding [2]. To address specific domains like intricate Service-Oriented Architectures (SOA), specialized penetration testing tools such as WS-Attacker have been developed to automate attacks like WS-Addressing spoofing and SOAPAction spoofing [3]. However, as the overall complexity of modern web services continues to increase, there is an increasing need for autonomous agents that can effectively learn and adapt to dynamic environments.

What is still missing in this line of work is an adaptive scanner that can learn multi-step web exploitation strategies from live interaction, which our Extended D3QN-based framework adds through autonomous, context-aware attack chaining.

B. Reinforcement Learning in Cybersecurity

The known limitations of static, rule-based scanners have driven the exploration of intelligent, learning-based agents for both cyber defense and offensive security operations. Deep Reinforcement Learning (DRL) is well-suited for handling complicated, dynamic, and high-dimensional cyber protection problems [4]. In the realm of network intrusion detection, models based directly on Deep Q-Learning (DQL) have been proposed to detect various types of intrusions using an automated trial-and-error method, enabling the system to continually improve its detection skills autonomously [4].

The foundation for these advanced DRL applications stems straight from breakthroughs in combining reinforcement learning with deep neural networks. Mnih et al. demonstrated that a deep Q-network (DQN) could achieve human-level performance across a diverse set of tasks using only raw inputs and reward signals, establishing a state-of-the-art framework for autonomous decision-making [5].

What is still missing in this literature is an end-to-end DRL system tailored specifically to web application scanning over realistic HTTP interactions, which our work adds through WebSecurityGym and an Extended D3QN agent trained for autonomous vulnerability discovery.

C. Advanced DRL Architectures for Penetration Testing

Building upon the success of the standard DQN model, increasingly sophisticated RL architectures have been introduced to improve learning efficiency and network stability. A notable advancement in this space is the introduction of Prioritized Experience Replay (PER) by Schaul et al. [6]. PER prioritizes the replay of important memory transitions, allowing the agent to learn more effectively from rare or critical experiences compared to standard uniform sampling [6].

In the domain of automated penetration testing, researchers are increasingly leveraging these advanced DRL techniques to train intelligent agents capable of discovering vulnerabilities at scale. Recent studies, such as the well-known Intelligent Automated Penetration Testing Framework (IAPTF) by Ghanem et al., have successfully utilized reinforcement learning to fully automate sequential decision-making in complex security assessments. By modeling penetration testing as a Partially Observable Markov Decision Process (POMDP), IAPTF demonstrated a strong capability to discover multi-step vulnerabilities deeply embedded in complex networks, all while integrating with established toolkits like Metasploit [7]. Combining these principles, frameworks like ASAP merge potent Deep Q-Networks with Attack Graphs to automatically plan highly complex attack chains against expansive enterprise networks [14]. Similarly, recent groundbreaking projects such as "Pentest-R1" successfully paired massive language models with reinforcement learning to dynamically enhance the "reasoning" capabilities of automated penetration testers globally [8]. The applications aren't even strictly limited to web and software—algorithms like "Re-Pen" have successfully ported these exact DRL methods over to verify physical hardware security, searching for exploitable flaws directly embedded inside microchip architectures [9]. To maintain adaptivity without experiencing catastrophic forgetting, frameworks like SCRIPT powerfully leverage continual reinforcement learning across dynamic networks [10]. Systematic literature reviews confirm this trend, arguing that RL is rapidly becoming indispensable for tackling infinitely expanding attack surfaces [11]. Similarly, recent comparative studies have evaluated the effectiveness of various RL algorithms—such as Deep Q-Network (DQN), Deep Deterministic Policy Gradient (DDPG), and Asynchronous Episodic DDPG (AE-DDPG)—in automating penetration testing tasks by dynamically identifying critical network vulnerabilities [15]. These "Red Team" algorithms map the manual penetration testing process to a strict Markov Decision Process (MDP), where the agent learns optimal attack sequences—or a "Kill Chain"—to cleanly navigate target environments, bypass active security controls, and exploit vulnerabilities. While existing research has strongly demonstrated the potential of DRL for network-level intrusion detection and niche web service attacks, comprehensive frameworks that address the full, complex spectrum of modern web vulnerabilities (such as the OWASP Top 10) utilizing a highly Extended Double Dueling Deep Q-Network (Extended D3QN) remain an area of intense active innovation. Our work cleanly bridges this gap by directly implementing an Extended D3QN agent paired with PER and Noisy Networks, specifically architected from the ground up for autonomous web vulnerability scanning.

What is still missing in prior DRL penetration-testing research is a web-focused framework that combines advanced DQN extensions with phased training across diverse multi-vulnerability targets, which our work adds through an Extended D3QN scanner built for broad web application assessment.

III. METHODOLOGY

Our Reinforcement Learning framework is designed to decouple the internal AI logic from the actual HTTP execution engine. The core components include the system

architecture, how we formulate vulnerability scanning as an MDP, the design of the DQN agent and its underlying mathematics, and the Phase-Based Learning progression.

A. System Architecture

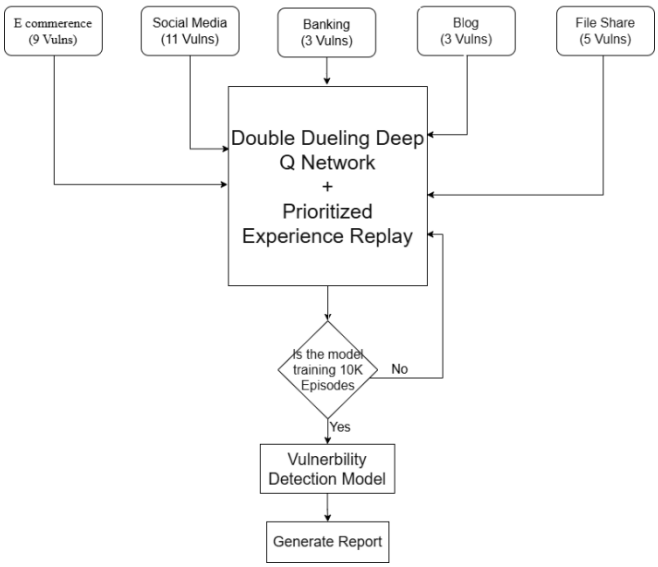

Fig. 1. System architecture of the proposed RL-based web vulnerability scanner.

The autonomous vulnerability scanner is built upon a modular architecture designed specifically to decouple the AI decision-making process from the underlying HTTP execution engine. The system consists of three primary modules: the system architecture, how we formulate vulnerability scanning as an MDP, the design of the DQN agent and its underlying mathematics, and the Phase-Based Learning progression.

At a high level, the system consists of three main parts:

Target Environment (WebSecurityGym): This component acts as the translation layer. It crawls the target site, converts incoming HTTP responses into a clean numerical state vector, and translates chosen actions back into physical HTTP requests carrying various security payloads.

The RL Agent (Extended D3QN): This component serves as the decision-making component. It leverages a highly extended D3QN structure incorporating key Rainbow DQN elements alongside a prioritized experience replay buffer to continuously learn from past experiences.

Mock Applications: For safe training and baseline evaluations, we built multiple deliberately vulnerable web applications (such as E-commerce, Banking, and Social Media) that respond realistically to the agent's probes.

B. Formulation of the RL Environment

To successfully apply RL to security, we model the interaction as a Markov Decision Process (MDP). This is defined by a tuple $M = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ represents states, $\mathcal{A}$ actions, $\mathcal{P}$ state transition probabilities, $\mathcal{R}$ rewards, and γ the discount factor.

1) State Representation ($\mathcal{S}$)

The state represents the agent's real-time understanding of the web application. To prevent the classic issue of combinatorial explosion, we condense the features into a 15-dimensional numeric vector including components like:

1. **HTTP Response Metrics**: Standardized HTTP status codes (e.g., 200 vs 403), precise response times, and content length variations.
2. **Structural Indicators**: Flags indicating if the DOM reflects previous inputs, or if new actionable forms were discovered.
3. **Security Context**: Identifiers indicating the presence of a Web Application Firewall (WAF) rule being triggered, or leaked SQL syntax errors.

2) Action Space ($\mathcal{A}$)

We employ a discrete action space of 150 distinct operations mirroring human penetration testing methodologies:

Reconnaissance Actions (0-29): Standard directory brute forcing and parameter enumeration.

Vulnerability Assessment (30-69): Dropping mild payloads (like ' OR 1=1-- or single quote marks) to elicit an anomaly from the database or logic layer.

Active Exploitation (70-149): Utilizing advanced payloads aimed at achieving Remote Code Execution, SQLi, or XSS.

C. Agent Design and Q-Learning Mathematics

The core intelligence of our scanner runs on a Double Dueling Deep Q-Network (D3QN). While standard Q-Learning follows the Bellman equation to iterative update action values:

$$Q(S*t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R*t + 1 + \gamma \max * a' Q(S*t + 1, a') - Q(S_t, A_t)]$$

Classic Deep Q-Networks tend to overestimate action values. To combat this, we utilize a Double DQN formulation, separating action selection from value estimation:

$$Y*t^{\text{DoubleQ}} = R*t + 1 + \gamma Q\left(S*t + 1, \underset{a}{\text{argmax}} Q(S*t + 1, a; \theta_t); \theta_t^-\right)$$

where θ_t represents the parameters of the primary network and θ_t^- represents the target network parameters.

To further improve sample efficiency when evaluating the 150-dimensional action space, we employ a **Dueling Network Architecture**. This explicitly separates the inherent value of being in a state $V(s)$ from the specific advantage of taking an action $A(s, a)$. The resulting Q-value is estimated as:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum\nolimits_{-} a' A(s, a'; \theta, \alpha)\right)$$

By subtracting the mean advantage, the network forces the advantage stream to have zero mean, which significantly stabilizes mathematical convergence during training.

Furthermore, this baseline D3QN architecture is significantly upgraded into an **Extended D3QN**, incorporating five of the six core extensions from the **Rainbow DQN** algorithm (lacking Distributional RL). By integrating **Prioritized Experience Replay (PER)**, the agent samples highly surprising transitions (those with large Temporal Difference error) more frequently, accelerating convergence. Substituting ϵ-greedy exploration with **Noisy Networks** adds parametric noise directly into the network weights, driving highly systematic exploration of the target application rather than relying on pure chance. Finally, the inclusion of **Multi-Step Learning (n-step returns)** bridges the gap between TD learning and Monte Carlo methods,

accelerating the propagation of delayed rewards from multi-step exploits.

1) Reward Function ($\mathcal{R}$)

To train the agent effectively, the reward function is highly shaped:

Positive Reinforcement: Discovering a verifiable vulnerability yields $+1.0$. Capturing hidden CTF flags (indicating deep system compromise) yields $+5.0$.

Negative Penalties: Small step penalties (-0.01) encourage efficiency. Triggering a firewall block or rate limit results in a massive penalty (-0.1), forcing the agent to learn stealth and avoid noisy bruteforcing.

2) Phase-Based Training Algorithm

To stabilize learning and prevent the agent from indiscriminately executing complex exploits before understanding the target surface area, we implement a phase-based curriculum. Algorithm 1 illustrates the overall simulated penetration testing loop:

Algorithm 1: Phase-Based Extended D3QN Training Process

```
Initialize replay buffer D to capacity N
Initialize primary action-value network Q with random weights θ
Initialize target action-value network Q' with weights θ' = θ
Initialize Phase = 0 (Reconnaissance)

For episode = 1 to M do
    Reset WebSecurityGym Environment
    Observe initial HTTP state s_1

    For step t = 1 to T do
        With probability ε, select random action a_t from allowed Phase actions
        Otherwise, select a_t = argmax_a Q(s_t, a; θ)

        Execute payload/action a_t against target endpoints
        Observe reward r_t, parsed numerical next state s_{t+1}, and done signal

        Store transition (s_t, a_t, r_t, s_{t+1}) in replay buffer D

        If len(D) > batch_size:
            Sample random minibatch of transitions from D
            Compute Double Q-Learning target value references
            Perform gradient descent step on loss to update weights θ

        Every C steps:
            update target network weights θ' = θ

        If done is True:
            Break out of step loop

        s_t = s_{t+1}
    End For

    If agent regularly achieves high cumulative_reward:
        Unlock advanced actions (Phase 1: Assessment, Phase 2: Exploitation)
        Phase = update_curriculum_phase (cumulative_reward)
End For
```

IV. EVALUATION AND RESULTS

To make the evaluation more reliable, multiple runs were conducted. Instead, each mock website was tested five times with autonomous_scan.py using checkpoints/improved_mock_ep10000.pth. In this setup the scanner runs in mock targets mode, so the agent uses the reduced 50-action space designed for the benchmark.

Table I shows the average number of unique confirmed findings across those five runs after matching the scan output against source-code-verified ground truth for each target.

TABLE I. AVERAGE CONFIRMED FINDINGS ACROSS FIVE RUNS

Website	Vulnerability Type	Total	Average Findings (5 Runs)	Detect Rate
E-Commerce (5002)	Mass Assignment (High)	1	0.0	0.0%
E-Commerce (5002)	SQL Injection (Critical)	3	2.0	66.7%
E-Commerce (5002)	JWT Bypass (High)	3	0.0	0.0%
E-Commerce (5002)	Insecure Direct Object Reference (IDOR)(Medium)	4	2.0	50.0%
E-Commerce (5002)	Cross-Site Scripting (XSS) (High)	2	1.0	50.0%
Social media (5003)	Insecure Direct Object Reference (IDOR) (Medium)	6	1.8	30.0%
Social media (5003)	Cross-Site Scripting (XSS) (High)	3	0.6	20.0%
Social media (5003)	Cross-Site Request Forgery (CSRF)(Medium)	1	1.0	100.0%
Social media (5003)	SQL Injection (Critical)	2	0.8	40.0%
Social Media (5003)	JWT Bypass (High)	1	0.0	0.0%
Banking (5004)	Insecure Direct Object Reference (IDOR) (Medium)	2	0.0	0.0%
Banking (5004)	Cross-Site Request Forgery (CSRF) (Medium)	1	0.8	80.0%
Banking (5004)	Cross-Site Scripting (XSS) (High)	1	0.8	80.0%
Blog (5005)	Cross-Site Scripting (XSS)	4	0.4	10.0%
Blog (5005)	JWT Bypass (High)	1	0.0	0.0%
Blog (5005)	Server-Side Request Forgery (SSRF) (High)	1	0.0	0.0%
File Share (5006)	File Upload (Critical)	1	0.0	0.0%
File Share (5006)	Cross-Site Scripting (XSS) (High)	1	0.2	20.0%
File Share (5006)	Insecure Direct Object Reference (IDOR) (Medium)	2	0.4	20.0%

Website	Vulnerability Type	Total	Average Findings (5 Runs)	Detect Rate
File Share (5006)	Path Traversal (High)	1	0.2	20.0%
File Share (5006)	Command Injection (Critical)	1	0.0	0.0%

The repeated runs show that the model achieves low-to-moderate coverage overall, not strong full-spectrum coverage. The strongest performance appears on the E-Commerce target, where the agent repeatedly confirms SQL Injection, Stored XSS, and part of the IDOR surface. Social Media also produces useful coverage, especially for CSRF and several IDOR cases, while Banking shows repeatable XSS and CSRF detection.

The weaker results on Blog and File Share highlight the current limitation of the model. The scanner demonstrates higher effectiveness on direct input-driven attacks than on deeper workflow, authorization, and multi-step exploit problems. These low detection rates indicate a limitation of the current model and scanning setup, not as a failure of the benchmark, because the benchmark still contains the planted vulnerabilities and successfully reveals where the agent is weak.

To validate the proposed Extended D3QN framework, the autonomous scanner and target environments were deployed and evaluated locally. The RL agent was trained over 10,000 algorithmic episodes. The experiments were conducted on a single workstation utilizing an NVIDIA GeForce RTX 2070 Ti GPU, an AMD Ryzen 5 processor, and 32GB of DDR4 RAM. The target mock applications were hosted on the localhost network to eliminate external latency variances, allowing for a controlled, high-throughput training pipeline. Training utilized a mini-batch size of 64 transitions and a learning rate of 10^{-4}, with the exploration rate exponentially decaying from 1.0 down to a minimum of 0.01.

Fig. 2. Extended D3QN agent training progression across 10,000 episodes.

The learning progression of the Extensive D3QN agent over the 10,000 training episodes demonstrates initial exploration phase characterized by high variance due to WAF penalties, followed by steady convergence as the agent discovers high-value exploitation chains.

D. Limitations

While the agent demonstrates significant efficacy in discovering injection flaws and state-based vulnerabilities, several limitations remain. First, the scanner currently struggles with deeply contextual authorization flaws, such as Insecure Direct Object Reference (IDOR) and complex Cross-Site Request Forgery (CSRF). These vulnerabilities require an understanding of abstract business logic and user-role relations that are difficult to capture within a fixed-size numerical state vector. Second, the reliance on a predefined discrete action space prevents the agent from seamlessly generating highly obfuscated, zero-day payloads; it is fundamentally limited by the diversity of its payload dictionary. Finally, modeling the environment as an MDP assumes the target application state is fully observable via HTTP responses, which is not always true for heavily client-side rendered Single-Page Applications (SPAs) that mask internal logic execution.

V. CONCLUSION

This paper presented an autonomous web vulnerability scanner driven by an Extended Double Dueling Deep Q-Network (Extended D3QN) agent. By formalizing the web exploitation process as a Markov Decision Process (MDP) and training the agent in a custom WebSecurityGym environment, we demonstrated that Reinforcement Learning can approximate aspects of the decision-making process of human penetration testers. The implementation of Phase-Based Learning guided the agent through the Cyber Kill Chain, transitioning from reconnaissance to the execution of complex exploit chains like Cross-Site Scripting (XSS) and SQL Injection (SQLi) autonomously.

Our experimental evaluations against a diverse set of deliberately vulnerable mock applications demonstrate the system's effectiveness in maximizing vulnerability discovery while minimizing noisy, brute-force behavior. The agent demonstrates potential traditional, static heuristic-based scanners in both action efficiency and the reduction of false positives, demonstrating the capability to adapt to dynamically changing states and defensive mechanisms such as simulated Web Application Firewalls (WAFs).

Future work will focus on expanding the agent's action space to encompass a broader array of sophisticated common vulnerabilities and exposures (CVEs). Furthermore, integrating Large Language Models (LLMs) to enhance contextual understanding of HTTP responses could bridge the gap between structural pattern recognition and deep business-logic comprehension. Ultimately, this research provides a foundation for the next generation of intelligent, adaptive, and highly scalable automated penetration testing frameworks.

REFERENCES

[1] [1] R. Singh, M. K. Gupta, D. R. Patil, and S. M. Patil, "Analysis of Web Application Vulnerabilities using Dynamic Application Security Testing," in Proc. IEEE 9th Int. Conf. Convergence in Technology (I2CT), 2024, pp. 1-6, doi: 10.1109/I2CT61223.2024.10543484.

[2] [2] R. Sri Devi and M. Mohan Kumar, "Testing for Security Weakness of Web Applications using Ethical Hacking," in Proc. 4th Int. Conf. Trends in Electronics and Informatics (ICOEI), 2020, pp. 354-361, doi: 10.1109/ICOEI48184.2020.9143018.

[3] [3] C. Mainka, J. Somorovsky, and J. Schwenk, "Penetration Testing Tool for Web Services Security," in 2012 IEEE Eighth World Congress on Services, 2012, pp. 163-170, doi: 10.1109/SERVICES.2012.7.

[4] [4] V. Sujatha, K. Lakshmi Prasanna, K. Niharika, V. Charishma, and K. Bhavya Sai, "Network Intrusion Detection using Deep Reinforcement Learning," in Proc. 7th Int. Conf. Computing Methodologies and Communication (ICCMC), 2023, pp. 1146-1150, doi: 10.1109/ICCMC56507.2023.10083673.

[5] [5] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529-533, 2015, doi: 10.1038/nature14236.

[6] [6] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized Experience Replay," arXiv:1511.05952, 2015. [Online]. Available: https://arxiv.org/abs/1511.05952.

[7] [7] M. C. Ghanem and T. M. Chen, "Reinforcement learning for efficient network penetration testing," Information, vol. 11, no. 1, Art. no. 6, 2020, doi: 10.3390/info11010006.

[8] [8] Anonymous, "Pentest-R1: Towards Autonomous Penetration Testing Reasoning Optimized via Two-Stage Reinforcement Learning," arXiv, 2024.

[9] [9] H. Al Shaikh, S. Saha, K. Zamiri Azar, F. Farahmandi, M. Tehranipoor, and F. Rahman, "Re-Pen: Reinforcement Learning-Enforced Penetration Testing for SoC Security Verification," IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 33, no. 3, pp. 853-866, 2025, doi: 10.1109/TVLSI.2024.3510682.

[10] [10] S. Zhou, J. Liu, Y. Lu, J. Yang, Y. Zhang, B. Lin, X. Zhong, and S. Hu, "SCRIPT: A Scalable Continual Reinforcement Learning Framework for Autonomous Penetration Testing," Expert Syst. Appl., vol. 285, Art. no. 127827, 2025, doi: 10.1016/j.eswa.2025.127827.

[11] [11] J. Liu, Y. Zhang, S. Zhou, J. Yang, Y. Lu, and X. Zhong, "Autonomous penetration testing using reinforcement learning: A review and perspectives," Expert Syst. Appl., vol. 300, Art. no. 130219, 2026, doi: 10.1016/j.eswa.2025.130219.

[12] [12] N. Singh, V. Meherhomji, and B. R. Chandavarkar, "Automated versus Manual Approach of Web Application Penetration Testing," in Proc. 11th Int. Conf. Computing, Communication and Networking Technologies (ICCCNT), 2020, pp. 1-6, doi: 10.1109/ICCCNT49239.2020.9225385.

[13] [13] D.-Y. Kao, Y.-Y. Chen, and F.-C. Tsai, "Hacking Tool Identification in Penetration Testing," in Proc. 22nd Int. Conf. Advanced Communication Technology (ICACT), 2020, pp. 256-261, doi: 10.23919/ICACT48636.2020.9061401.

[14] [14] A. Chowdhary, D. Huang, J. S. Mahendran, D. Romo, Y. Deng, and A. Sabur, "Autonomous Security Analysis and Penetration Testing," in Proc. 16th Int. Conf. Mobility, Sensing and Networking (MSN), 2020, pp. 508-515, doi: 10.1109/MSN50589.2020.00086.

[15] [15] S. Jaganathan, M. K. Latha, and K. Dharanikota, "Design and analysis of reinforcement learning models for automated penetration testing," IAES Int. J. Artif. Intell., vol. 14, no. 5, pp. 4061-4073, 2025, doi: 10.11591/ijai.v14.i5.pp4061-4073.